

# GNN-MLS: Signal Routing in Mixed-Node 3D ICs through GNN-Assisted Metal Layer Sharing

Jiawei Hu<sup>1</sup>, Pruek Vanna-iampikul<sup>3</sup>, Zhen Zhuang<sup>2</sup>, Tsung-Yi Ho<sup>2</sup>, and Sung Kyu Lim<sup>1</sup>

<sup>1</sup>School of ECE, Georgia Institute of Technology, Atlanta, GA

<sup>2</sup>The Chinese University of Hong Kong; <sup>3</sup>Department of Electrical Engineering, Burapha University, Chonburi, Thailand;

{jiawei.hu, limsk}@gatech.edu; zhuangzhen1995@gmail.com; tyho@cse.cuhk.edu.hk; pruek.va@eng.buu.ac.th;

**Abstract**—Native 3D Integrated Circuit (3D IC) design offers enhanced performance and density but faces challenges in signal routing due to limited true 3D EDA tool support. Pseudo-3D flows bridge this gap but lack cross-tier optimization, critical for both mixed-node and homogeneous designs. Metal Layer Sharing (MLS) addresses this by enabling cross-tier routing co-optimization but risks timing degradation if not applied strategically. Additionally, MLS creates open connections in hybrid-bonded 3D ICs, making chips untestable. We propose GNN-MLS, a Graph Neural Network-based framework for precise MLS net selection, combined with a tailored DFT solution for robust testability. Experiments show GNN-MLS reduces timing violations by 79% and improves WNS and TNS by 81% and 94%, and moves designs closer to true 3D ICs.

## I. INTRODUCTION

Three-dimensional Integrated Circuits (3D ICs) are a promising solution to extend Moore's Law by providing increased device density and performance through vertical integration. However, current Electronic Design Automation (EDA) tools, primarily designed for 2D ICs, face significant limitations in supporting true 3D integration. While dedicated 3D flows have been proposed for placement [1]–[3], true 3D routing remains underexplored, and these methods often fall short of achieving manufacturable quality. As a result, pseudo-3D flows, such as sequential-2D [4], Memory-on-Logic (MoL) [5], and pin-3D [6], have emerged as practical alternatives, leveraging 2D EDA tools to achieve commercially viable 3D designs. However, these flows fail to exploit fine-grained cross-tier optimization, leaving untapped opportunities for performance improvements [7]–[10].

Metal Layer Sharing (MLS) offers a promising pathway to bridge the gap between pseudo-3D and true 3D capabilities by enabling cross-tier routing co-optimization through 2D EDA tools. MLS facilitates the use of routing layers across tiers, as depicted in Figure 1, unlocking additional paths and enhancing timing performance. However, state-of-the-art (SOTA) approaches to MLS rely on heuristic strategies with limited net-level control, leading to indiscriminate MLS application that can degrade timing performance. For example, in our experiments with the MAERI 128PE design [11], SOTA MLS applied to specific nets worsened slack (e.g., net n146095 had slack degraded to -48ps), whereas disabling MLS improved slack to -45ps as shown in Table I. These findings underscore the need for more targeted MLS strategies. Figure 2 illustrates that while SOTA improves timing with MLS, our proposed method achieves far superior results.

Additionally, MLS introduces testability challenges, particularly in hybrid-bonded 3D ICs. Open connections created during fabrication render critical signals unobservable or uncontrollable, leading to non-testable designs. As illustrated in Figure 3, these open connections significantly affect testability in hybrid-bonded designs. Addressing these issues is crucial for realizing MLS's potential in cross-tier co-optimization and advancing pseudo-3D designs toward true 3D integration.

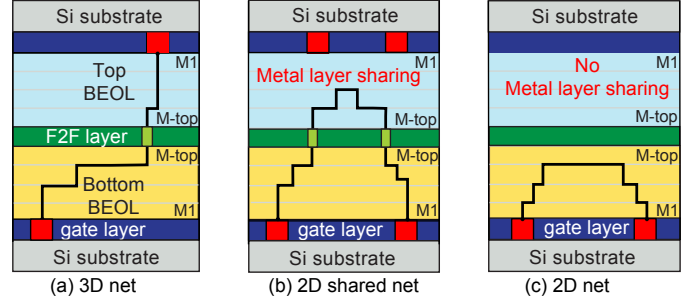


Fig. 1: Illustration of 3-d net, 2d-shared net, and 2d-net.

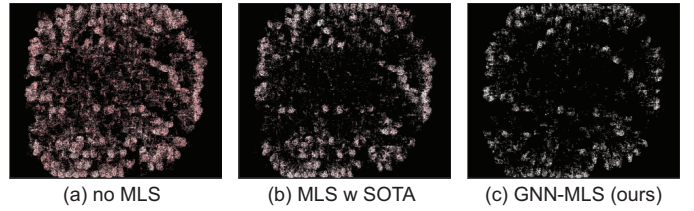


Fig. 2: Timing Violation Points: Registers with violations are highlighted. No MLS has the most violations, SOTA reduces violations by 68%, and GNN-MLS achieves an 80% reduction.

To address these challenges, we propose **GNN-MLS**, a Graph Neural Network-assisted framework for fine-grained, net-level MLS decision-making focused on optimizing timing in 3D ICs. The core challenge lies in evaluating the timing impact of MLS on each net, which traditionally requires exhaustive Static Timing Analysis (STA) across all configurations—a computationally prohibitive process. GNN-MLS eliminates this need by leveraging a novel approach that converts the hyper-edge nature of nets into a node-centric problem, enabling efficient graph-based modeling of timing paths.

Using a transformer-based encoder, GNN-MLS captures the intricate interdependencies among nets along critical timing paths, ensuring precise MLS decisions that prioritize timing improvements. Additionally, we introduce a tailored Design-for-Test (DFT) solution to address testability challenges in hybrid-bonded 3D ICs, ensuring signal observability and controllability across tiers with minimal overhead. By strategically focusing on timing and testability, GNN-MLS advances pseudo-3D flows toward the performance and reliability of true 3D ICs, offering a scalable and practical solution for next-generation integration.

In this work, we propose a comprehensive framework for cross-tier routing co-optimization in 3D ICs, combining GNN-MLS for MLS decision-making, DFT for testability, and PDN design for heterogeneous stacking. Our main contributions are as follows:

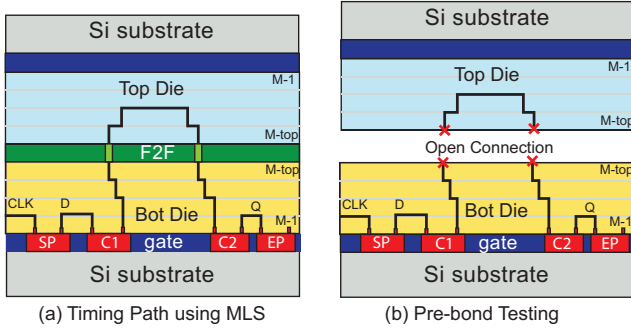


Fig. 3: Testability Issue in Hybrid-Bonded Design with MLS: Each MLS induces an open connection during individual die fabrication, which makes the chip untestable. The timing path consists of start-point (SP), combinational logic (C1-2), and endpoint (EP).

- 1) We propose GNN-MLS, integrating hyper-graph conversion, a Transformer for net interdependencies, contrastive pretraining, and supervised fine-tuning for MLS decision-making.
- 2) We design two DFT strategies for MLS in hybrid-bonded 3D ICs, ensuring high test coverage with minimal overhead.
- 3) We develop a multi-power domain framework and a PDN for mixed-node 3D ICs to support heterogeneous integration experiments.
- 4) Our framework achieves up to 75% improvement in WNS, 94% in TNS, and a 75% reduction in timing path violations compared to sequential 2D designs.

## II. MOTIVATION

### A. Why MLS and What are its Limitations?

Metal Layer Sharing (MLS) is a critical innovation for routing efficiency in 3D ICs, addressing key challenges such as congestion and timing optimization. By enabling nets to utilize BEOL layers across tiers, MLS provides additional routing resources and unlocks new pathways for critical connections, which are otherwise constrained in 2D ICs. For example, in the MAERI architecture with 16PE, MLS improves critical path slack from -76 ps without MLS to -18 ps with selective MLS, demonstrating its potential to enhance timing and overall performance.

However, current EDA tools do not model or systematically implement MLS. The only systematic approach to applying MLS is through iterative STA, which evaluates the timing impact of MLS on a net-by-net basis. This process involves repeated disconnection, rerouting, and slack recalculation, making it computationally infeasible for large designs. Furthermore, MLS introduces severe testability challenges in hybrid-bonded 3D ICs, as inter-tier connections disrupt signal propagation and render dies non-testable. These limitations restrict the practical adoption of MLS despite its clear performance benefits.

To address these issues, we propose a comprehensive framework that combines machine learning-based MLS decision-making to eliminate exhaustive STA evaluations with tailored DFT solutions to ensure robust testability. This approach significantly improves the efficiency and practicality of MLS in modern 3D ICs.

### B. Graph Neural Networks for Metal Layer Sharing

Metal Layer Sharing (MLS) decisions are essential for optimizing timing in 3D ICs, but systematically applying MLS presents significant challenges. Slack, a key timing metric, depends on the cumulative delays of all nets in a timing path, where changes to one

TABLE I: Metal layer sharing (MLS) impact on slack with single net of MAERI 128PE using the heterogeneous 3D integration (16nm logic, 28nm memory).

| Net name | before MLS |             | after MLS  |                     |
|----------|------------|-------------|------------|---------------------|
|          | slack (ps) | used metals | slack (ps) | used metals         |
| n480132  | -62        | M1-6(bot)   | -45        | M1-6(bot)+M5-6(top) |
| n146095  | -45        | M1-4(bot)   | -48        | M1-6(bot)+M6(top)   |

net propagate through others, creating intricate interdependencies. The only systematic approach for evaluating MLS decisions is through Static Timing Analysis (STA), which requires exhaustively exploring the entire MLS configuration space. This process involves iterative disconnection, rerouting, and recalculating slack for each net, making it computationally prohibitive for large designs and delaying cross-tier routing optimization.

To address the challenges of modeling timing paths, we leverage Graph Neural Networks (GNNs) to represent timing paths as directed acyclic graphs (DAGs), where nodes correspond to devices and edges to nets. GNNs provide a natural framework for propagating information across the graph, capturing local dependencies and structural relationships within the path. However, timing paths exhibit complex interdependencies that extend beyond immediate neighbors, requiring the ability to model both local interactions and long-range dependencies effectively. To tackle this, we integrate the Transformer [12] architecture into the GNN framework. Transformers, with their self-attention mechanism, dynamically weigh interactions across nodes, enabling them to model long-range relationships along the path. This ability to capture both sequential and global interactions complements the GNN's structural modeling, making the combined framework particularly powerful for understanding the cascading impact of MLS decisions and optimizing timing paths.

A significant challenge in implementing MLS is the scarcity of labeled data, as generating MLS labels through STA is computationally expensive and impractical at scale. To overcome this, we employ self-supervised learning with Deep Graph InfoMax (DGI) [13] to pretrain the Transformer. DGI maximizes mutual information between global path representations and local node embeddings, enabling the extraction of meaningful features from unlabeled timing path data. These pre-trained embeddings are then fine-tuned using a limited labeled dataset, which includes 500 timing paths each from the heterogeneous A7 single-core and MAERI 128PE designs, as well as 500 timing paths from the corresponding homogeneous configurations. This approach significantly reduces reliance on STA while ensuring efficient and accurate MLS predictions.

By combining the sequential modeling capabilities of Transformers, the graph-structured representation of GNNs, and the data efficiency of DGI, our framework addresses the challenges of interdependencies, computational overhead, and data scarcity. This enables early, efficient, and accurate MLS decision-making, advancing cross-tier routing optimization in 3D IC designs.

## III. GNN-MLS ALGORITHMS

Cross-tier co-optimization in 3D ICs with Metal Layer Sharing (MLS) poses significant computational challenges. Evaluating the timing impact of MLS relies on post-routing Static Timing Analysis (STA), which requires iterating through multiple MLS configurations. This process is infeasible for large designs due to its high computational cost. To address this, we propose GNN-MLS, a machine learning-based framework that predicts MLS decisions early in the design flow. By moving MLS decisions to the routing stage, GNN-

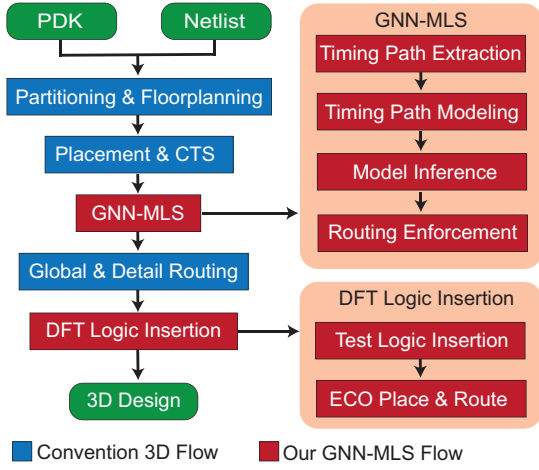


Fig. 4: GNN-MLS Design Flow

MLS reduces dependency on STA and enables efficient cross-tier routing optimization.

#### A. Design Flow

Our design flow integrates GNN-MLS to tackle timing optimization for monolithic and hybrid-bonded 3D ICs, while also ensuring testability in hybrid-bonded designs. Without MLS-specific tools, current methods rely entirely on post-routing STA to determine MLS decisions, restricting EDA tools from leveraging MLS during earlier stages. This late-stage dependency results in limited optimization opportunities and increased computational overhead. By enabling MLS predictions during routing, GNN-MLS integrates seamlessly into the design flow, facilitating timing optimization and ensuring testability at an earlier stage.

Figure 4 illustrates the key stages of the flow:

- 1) **MLS Decision-Making with GNN-MLS:** The GNN-MLS module generates binary MLS decisions for individual nets, indicating whether MLS should be applied. These decisions identify whether MLS should be applied based on a learned node representations. By providing these decisions early, GNN-MLS enables efficient routing optimization, allowing EDA tools to consider MLS during cross-tier routing.
- 2) **Targeted Routing and DFT Integration:** Using the MLS decisions from GNN-MLS, targeted routing enforces selective metal layer sharing, ensuring that only specific nets share metal layers across tiers to maximize timing performance. Following routing, custom Design-for-Test (DFT) logic is integrated to ensure testability. This addresses challenges unique to MLS-enabled designs, ensuring that cross-tier connections remain verifiable.

This design flow reduces the computational cost of iterative STA while achieving practical cross-tier co-optimization and addressing testability challenges.

#### B. Timing Path Modeling

Evaluating the timing impact of MLS on each net before routing is essential to optimize timing performance and eliminate iterative steps in the design flow. This fine-grained decision problem can be formulated as:

**Problem 1 (Net-Level MLS Decision):** Given a netlist  $\mathcal{N}$  and a timing path set  $\mathcal{P}$ , each net in a timing path should decide whether it uses MLS to maximize the slack of its timing path.

TABLE II: Hand-crafted Features used in our GNN-MLS.

| Features            | Description                                | Unit     |
|---------------------|--------------------------------------------|----------|
| cell location (x,y) | location of the cells                      | $\mu m$  |
| cell delay          | delay of the cells                         | $ps$     |
| pin capacitance     | pin capacitance of the output pin          | $pF$     |
| wirelength          | early-global routing wirelength of the net | $\mu m$  |
| wire capacitance    | estimated wire capacitance of the net      | $pF$     |
| wire resistance     | estimated wire resistance of the net       | $\Omega$ |

The optimal slack for each path  $p_i$  is given by:

$$\text{slack}_{\text{opt}}(p_i) = \text{slack}_{2D}(p_i) + f(\delta(n_1), \delta(n_2), \dots, \delta(n_{|p_i|})), \quad (1)$$

where  $\text{slack}_{2D}(p_i)$  represents the slack of  $p_i$  in the sequential-2D design (no-MLS).  $\delta(n_i)$  is a binary variable to identify if the net  $n_i \in \mathcal{N}$  uses MLS.  $f(\cdot)$  represents the slack variation caused by the MLS decision compared with the sequential-2D design. The optimization objective then becomes:

$$\max_{\delta(n_i) \in \{0,1\}} \sum_{p_i \in \mathcal{P}} \text{slack}_{\text{opt}}(p_i) \quad (2)$$

However, solving this optimization directly is infeasible due to the exponential number of MLS configurations and the need for STA evaluations to compute  $f(\cdot)$ . Instead, GNN-MLS approximates this solution by learning to predict the optimal MLS decisions  $\delta(n_i)$  based on timing path dependencies. By framing the problem in this manner, GNN-MLS allows efficient decision-making while indirectly optimizing the total slack objective.

We propose a timing path modeling methodology tailored to the unique characteristics of timing paths, as shown in Figure 5. Circuits are first modeled as directed acyclic graphs (DAGs) based on signal transmission. However, multi-pin nets, which connect three or more pins and interconnect multiple devices, introduce significant challenges for MLS decision-making due to their complexity.

To simplify this, each multi-pin net is treated as a hyperedge with a single source node, corresponding to the output pin in the netlist. This transformation allows net-related MLS decisions to be reformulated as node-specific decisions, effectively combining edge features with node features. Graph Transformer architectures then process these features, transforming the initial node attributes into informative embeddings for accurate MLS predictions. The fused features are shown in Table II.

#### C. Graph Transformer-Based MLS Decisions

The Transformer architecture plays a central role in GNN-MLS, as it effectively captures the interdependencies among MLS decisions across nets within a timing path. The slack improvement for a timing path depends on how MLS is applied to all nets along the path. This interdependence, combined with the sequential nature of timing paths, necessitates a model that can capture both local and global relationships. The entire graph Transformer-based framework is shown in Figure 5.

Traditional GNNs, which aggregate information from immediate neighbors, are insufficient for this task because they rely on fixed neighborhood structures. The Transformer's self-attention mechanism dynamically weighs interactions between all nodes in a path, regardless of distance, making it ideal for capturing long-range dependencies. Additionally, the sequential nature of timing paths is preserved through positional encodings, which ensure that the order of nodes is explicitly encoded in the model. To balance expressiveness and efficiency, the proposed Transformer architecture has three layers. Each layer consists of a three-head self-attention

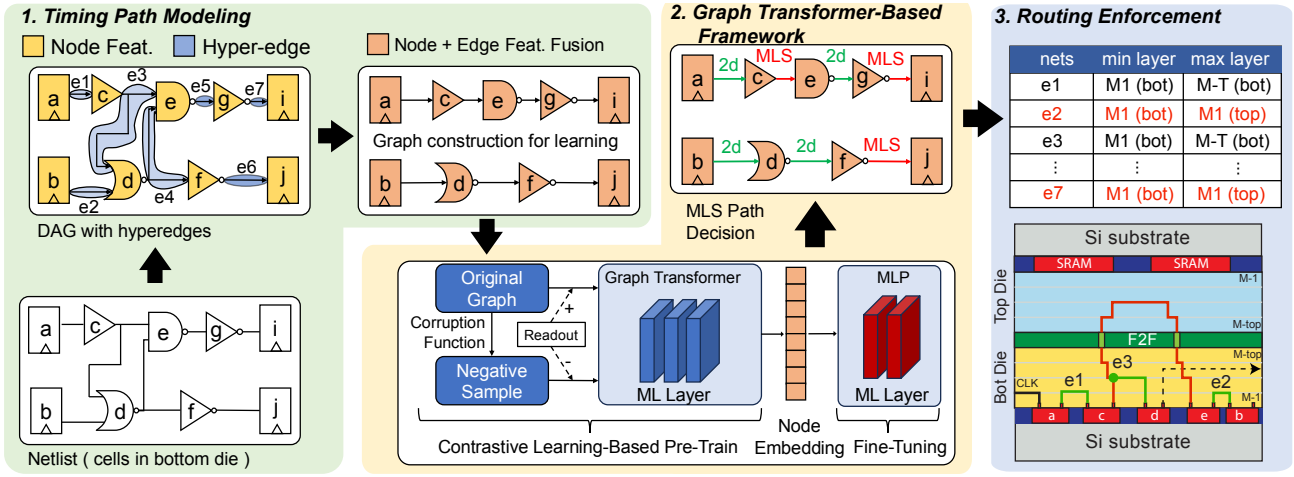


Fig. 5: The GNN-MLS framework converts netlist into hypergraph. It embeds hyperedge features into output-pin-associated nodes and extracts timing paths. The Graph Transformer, pretrained with DGI, processes these paths. MLP fine-tunes the embeddings to make MLS decisions.

mechanism. This architecture is tailored to the complexity of MLS decision-making, addressing the combinatorial dependencies among nets while maintaining computational efficiency.

Obtaining labeled data for MLS decisions, such as STA-based ground truth, is computationally expensive. To address this, we use Deep Graph Infomax (DGI) for self-supervised pretraining, allowing the model to learn effective node embeddings from unlabeled timing path graphs. DGI maximizes the mutual information between global path representations  $g(Y)$  and local node embeddings  $v$ , ensuring that embeddings capture both structural and timing-critical dependencies. Negative sampling, achieved by perturbing node features, enhances robustness by forcing the model to distinguish true node-path relationships from noise. We adopt sigmoid function  $\sigma$  to map inner product to probability and aid training stability. The DGI loss for a graph  $Y$  is defined as:

$$I(Y) = \sum_{v \in Y} \log(\sigma(\langle v, g(Y) \rangle)) + \sum_{v^* \in C(Y)} \log(\sigma(\langle v^*, g(Y) \rangle)) \quad (3)$$

Following DGI pretraining, the Transformer-generated embeddings are fine-tuned using a two-layer Multi-Layer Perceptron with collected labeled data. The MLP maps each node embedding to a binary decision  $\delta(n_i)$ , indicating whether MLS should be applied. The entire training scheme is shown in Algorithm 1.

#### D. DFT Strategy for MLS-Enabled Designs

While Metal Layer Sharing (MLS) significantly improves 3D routing by enabling cross-die connections, it introduces critical testability challenges. Each MLS connection creates an open circuit, disrupting signal propagation and preventing observability and controllability across dies, as shown in Figure 3. This renders upstream signals non-observable and downstream signals non-controllable, making MLS-enabled designs non-testable in hybrid-bonded 3D ICs. Conventional Design-for-Test (DFT) strategies fail to address these challenges, necessitating tailored solutions for MLS testability.

We propose two targeted DFT strategies designed for MLS connections, inserted post-routing to align with their precise locations. The first method, illustrated in Figure 6(a), adds a MUX at the MLS connection, toggling between functional and test modes. In

#### Algorithm 1: Training Process for MLS Decision-Making

**Input:** Set of timing path graphs  $\{Y_i\}_{i=1}^M$ ; node features  $X$ ; binary MLS labels for fine-tuning.

**Output:** Trained Transformer and MLP.

- DGI Pretraining:**
- for each timing path graph  $Y_i$  do**
- Compute node embeddings  $v$  using graph Transformer.
- Generate global summary vector  $g(Y_i)$ .
- Create perturbed graph  $C(Y_i)$  for negative samples  $v^*$ .
- Compute DGI loss to maximize mutual information.
- MLP Fine-Tuning:**
- for each labeled timing path graph  $Y_i$  do**
- Pass DGI-pretrained node embeddings by a 2-layer MLP.
- Train MLP on labels to predict MLS decisions  $\delta(n_j)$ .

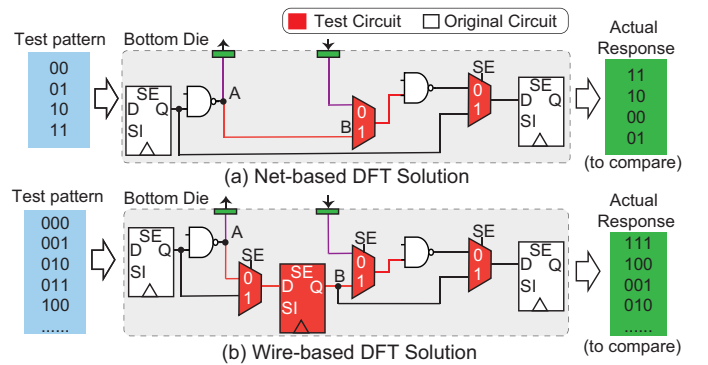


Fig. 6: Two DFT solution for MLS: (a) Net-based DFT where fault can be detected if either outgoing and incoming signal from F2F pads are incorrect, (b) Wire-based DFT where the mechanism can detect if outgoing or incoming F2F pads are fault.

test mode, the scan chain redirects signal flow across the MLS path, enabling observability and controllability. The second method, depicted in Figure 6(b), introduces a scan flip-flop (FF) at the MLS connection, improving test coverage by registering the upstream

| DFT method     | Total fault | Detected fault | WNS   |
|----------------|-------------|----------------|-------|
| Net-based DFT  | 444,296     | 438,152        | -21ps |
| Wire-based DFT | 444,346     | 438,276        | -23ps |

TABLE III: Two DFT approaches comparison for MLS nets: (a) net-based MLS DFT, (b) wire-based MLS DFT.

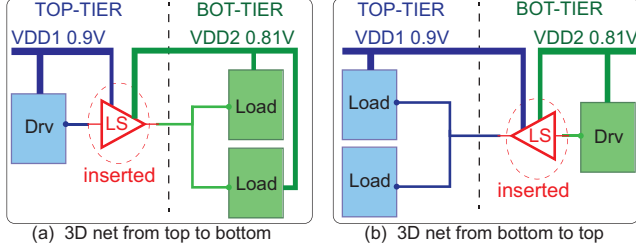


Fig. 7: Power Design for Hetero-Integration.

signal and driving the downstream signal during testing.

We evaluated these methods on the MAERI 16PE 4BW design with 16 MLS nets. As shown in Table III, while the MUX-based method minimizes additional logic, the FF-based method achieves superior fault coverage, detecting 94 more faults at the cost of 50 additional faults due to added logic. Post-routing ECO adjustments ensure that the timing impact of these solutions remains minimal. These results validate both approaches, with the FF-based method offering enhanced testability for MLS-enabled designs.

#### E. Power Delivery for Mixed-Node 3D ICs

In mixed-node 3D IC heterogeneous integration setup, we introduce a unique 3D power domain configuration, shown in Figure 7. The top-level operates at 0.9V, with the 28nm memory sub-domain also at 0.9V and the 16nm logic sub-domain at 0.81V. Level shifters are inserted for each 3D signal connection to manage voltage differences across tiers, facilitating reliable communication between the 16nm and 28nm technologies.

The PDN is implemented with specific width and pitch to ensure that the IR-drop of all design are within 10% of lowest VDD (0.81V) as defined in Table IV. The remaining routing resource are utilized for the 2D or MLS nets.

### IV. EXPERIMENTAL RESULTS

#### A. Experimental Setup

We evaluate the GNN-MLS framework on homogeneous and heterogeneous 3D integrations with Face-to-Face (F2F) bonding using three benchmarks: 128PE 32BW MAERI-ARCH, dual-core Cortex A7, and 256PE 64BW MAERI-ARCH. Heterogeneous designs use TSMC 28nm for the logic die and 16nm for the memory die, while homogeneous designs use 28nm for both dies. 3D Place-and-Route (PnR) is performed with Cadence DDI 23.12.000 (Innovus) and the Macro-3D flow [5]. F2F via parameters are configured as size  $0.5\mu m$ , pitch  $1.0\mu m$ , resistance  $0.5\Omega$ , and capacitance  $0.2fF$ .

We compare GNN-MLS with the state-of-the-art (SOTA) MLS technique [9] and a sequential 2D (no MLS) stacking method, analyzing timing, routing efficiency, and testability. Runtime, based on timing path depth, includes transformer-based training and inference. Results highlight GNN-MLS's advantages in both integration contexts.

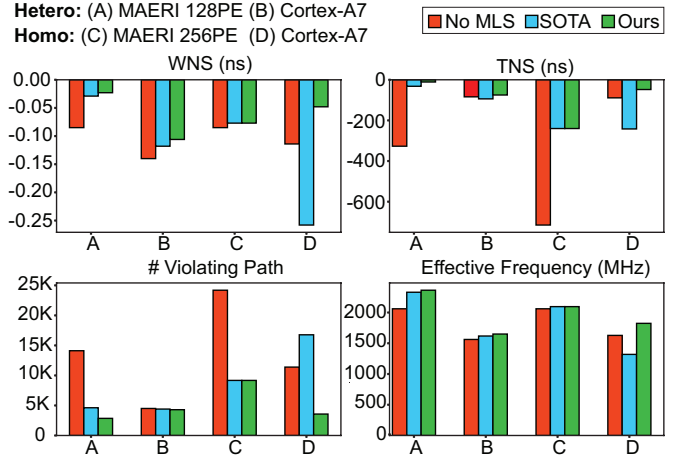


Fig. 8: Timing Metric Comparison on different design benchmark.

#### B. Results for Heterogeneous 3D ICs

In heterogeneous 3D ICs integrating 16nm logic with 28nm memory, GNN-MLS achieves substantial timing improvements (Figure 8, Table IV). For the MAERI 128PE design, which has a balanced logic-memory ratio, GNN-MLS reduces TNS to -11 ns from -32 ns (SOTA) and -327 ns (No MLS), and decreases violating paths to 2.8K from 4.6K (SOTA) and 14K (No MLS). WNS improves to -23 ps, outperforming both SOTA (-29 ps) and No MLS (-85 ps).

Similarly, in the A7 Dual-Core configuration, GNN-MLS reduces TNS to -75 ns versus -94 ns (SOTA) and -84 ns (No MLS). Violating paths decline to 4.2K from 4.4K (SOTA) and 4.5K (No MLS), with WNS improving to -106 ps compared to -118 ps (SOTA) and -140 ps (No MLS).

Notably, GNN-MLS applies MLS more selectively in heterogeneous designs. For MAERI 128PE, MLS usage drops to 2.4K nets from 9.5K (SOTA); for A7 Dual-Core, usage decreases to 2.6K nets from 3.5K (SOTA). This selective approach demonstrates GNN-MLS's adaptability, applying MLS strategically to maximize timing benefits and minimize overhead.

These results highlight GNN-MLS's ability to optimize timing in heterogeneous designs through targeted and efficient MLS application, meeting the specific demands of hybrid-bonded 3D ICs.

#### C. Results for Homogeneous 3D ICs

In homogeneous 3D ICs stacking 28nm memory on 28nm logic, GNN-MLS significantly improves timing (Figure 8, Table V). For MAERI 256PE, selected for balanced logic-memory ratio, GNN-MLS reduces TNS to -240 ns from -715 ns (SOTA) and -513 ns (No MLS). Violating paths drop to 9,173 from 24,195 (SOTA) and 16,037 (No MLS), and WNS improves to -0.077 ns, surpassing SOTA (-0.085 ns) and No MLS (-0.083 ns).

In the A7 Dual-Core design, GNN-MLS lowers TNS to -48 ns compared to -242 ns (SOTA) and -89 ns (No MLS). Violating paths decrease to 3,569 from 16,770 (SOTA) and 11,391 (No MLS), with WNS significantly improving to -0.048 ns versus -0.258 ns (SOTA) and -0.114 ns (No MLS).

GNN-MLS also expands MLS usage strategically in homogeneous designs. For MAERI 256PE, MLS coverage increases to 1,600 nets from 870 (SOTA); in A7 Dual-Core, it rises sharply to 73K nets from 8.4K (SOTA). This adaptive approach highlights GNN-MLS's ability to balance extensive MLS coverage with optimal timing.

TABLE IV: PPA metrics comparison between GNN-MLS, State-of-the-Art [9], and SOTA without MLS in Heterogeneous Integration. The frequency is in MHz, power is in mW, run time is in minutes. M-T represents the top-most layer of the memory die where W, P, and U denote the width, pitch, and utilization of the PDN on this metal layer. The highlighted entry denotes the best value among all design options.

|                       | 16nm Logic + 28nm Memory       |       |              |                                |       |              |
|-----------------------|--------------------------------|-------|--------------|--------------------------------|-------|--------------|
|                       | MAERI 128PE                    |       |              | A7 Dual-Core                   |       |              |
|                       | No MLS                         | SOTA  | Ours         | No MLS                         | SOTA  | Ours         |
| BEOL                  | 6+6                            |       |              | 8+8                            |       |              |
| Target Freq.          | 2,500                          |       |              | 2,000                          |       |              |
| FP (mm <sup>2</sup> ) | 0.38                           |       |              | 1.11                           |       |              |
| WL (m)                | 5.23                           | 5.18  | <b>5.16</b>  | <b>7.60</b>                    | 8.30  | 8.10         |
| WNS (ps)              | -85                            | -29   | <b>-23</b>   | -140                           | -118  | <b>-106</b>  |
| TNS (ns)              | -327                           | -32   | <b>-11</b>   | -84                            | -94   | <b>-75</b>   |
| #Vio. Paths           | 14,000                         | 4,600 | <b>2,800</b> | 4,500                          | 4,400 | <b>4,200</b> |
| #MLS Nets             | 0                              | 9,500 | 2,370        | 0                              | 3,542 | 2,621        |
| Run-Time              | -                              | -     | 20           | -                              | -     | 15           |
| Pwr                   | 1,472                          | 1,404 | <b>1,389</b> | <b>1,008</b>                   | 1,061 | 1,052        |
| IR-drop (%)           | 10.00                          | 9.50  | 9.40         | 1.90                           | 2.00  | 1.98         |
| M-T: W/P/U            | 2.00 $\mu$ m / 7 $\mu$ m / 14% |       |              | 2.70 $\mu$ m / 9 $\mu$ m / 30% |       |              |
| L.S Pwr               | 40                             | 45    | 46           | 31                             | 32    | 33           |
| Eff. Freq.            | 2,061                          | 2,330 | <b>2,363</b> | 1,562                          | 1,618 | <b>1,650</b> |

TABLE V: PPA metrics comparison between GNN-MLS, State-of-the-Art [9], and SOTA without MLS in Homogeneous Integration. The frequency is in MHz, power is in mW, run time is in minutes. The highlighted entry denotes the best value among all design options.

|                      | 28nm Logic + 28nm Memory |       |              |              |             |              |
|----------------------|--------------------------|-------|--------------|--------------|-------------|--------------|
|                      | MAERI 256PE              |       |              | A7 Dual-Core |             |              |
|                      | No MLS                   | SOTA  | Ours         | No MLS       | SOTA        | Ours         |
| Target Freq.         | 2500                     |       |              | 2000         |             |              |
| FP(mm <sup>2</sup> ) | 1.42                     |       |              | 1.11         |             |              |
| WL.(m)               | <b>14.5</b>              | 14.6  | 15.5         | <b>14.5</b>  | 12.1        | 11.2         |
| WNS (ps)             | -83                      | -85   | <b>-77</b>   | -114         | -258        | <b>-48</b>   |
| TNS (ns)             | -513                     | -715  | <b>-240</b>  | -89          | -242        | <b>-48</b>   |
| #Vio. Paths          | 16K                      | 24K   | <b>9K</b>    | 11K          | 16K         | <b>3.5K</b>  |
| #MLS Nets            | 0                        | 870   | 1.6K         | 0            | 8.4K        | 73K          |
| Run-Time             | -                        | -     | 35           | -            | -           | 15           |
| Pwr.(mW)             | <b>4680</b>              | 4747  | 4804         | 1425         | <b>1412</b> | 1442         |
| Eff. Freq.           | 2,070                    | 2,061 | <b>2,096</b> | 1,628        | 1,319       | <b>1,824</b> |

These results confirm GNN-MLS's effectiveness in enhancing timing through targeted MLS application in homogeneous designs.

#### D. Design-for-Test Results

Hybrid-bonded 3D ICs face testability issues from open MLS connections, affecting signal observability. We introduce a Design-for-Test (DFT) solution using scan flip-flops at critical MLS points with power gating for minimal overhead. DFT is applied to No MLS and GNN-MLS designs (excluding SOTA due to impractical probe pad needs).

As shown in Table VI, the combined GNN-MLS and DFT framework delivers robust testability and timing improvements. MAERI 128PE achieves 98.38% coverage, 75% fewer violating paths and WNS, 94% lower TNS, and a 15% frequency increase. A7 Dual-Core maintains 97.49% coverage, reduces violating paths by 17%, TNS by 10%, WNS by 5%, and frequency increases by 4.3%.

These results confirm the framework effectively addresses MLS testability and enhances timing in hybrid-bonded 3D ICs.

#### E. Power Delivery Network Design Results

We integrate the PDN into the heterogeneous 3D design, summarized in Table IV, carefully selecting PDN width and spacing to main-

TABLE VI: Comparison of MLS in Testable Designs between GNN-MLS and State-of-the-Art (28nm / 16nm Technology Node). The frequency is in MHz, power is in mW, run time is in minutes. The highlighted entry denotes the best value among all design options.

|                        | MAERI 128PE |                      | A7 Dual-Core |                     |
|------------------------|-------------|----------------------|--------------|---------------------|
|                        | No MLS      | GNN-MLS              | No MLS       | GNN-MLS             |
| Target Freq.           | 2500        | 2500                 | 2000         | 2000                |
| FP. (mm <sup>2</sup> ) | 0.38        | 0.38                 | 1.11         | 1.11                |
| WL. (m)                | 5.95        | <b>5.93 (0.3%)</b>   | 9.40         | <b>9.30 (1.1%)</b>  |
| Test Cover.(%)         | 98.25       | <b>98.38 (0.1%)</b>  | 97.32        | <b>97.49 (0.2%)</b> |
| WNS (ps)               | -86         | <b>-21 (75%)</b>     | -159         | <b>-132 (17%)</b>   |
| TNS (ns)               | -358.00     | <b>-20.00 (94%)</b>  | -112.00      | <b>-76.00 (32%)</b> |
| #Vio. Paths            | 15,321      | <b>3,766 (75%)</b>   | 6,055        | <b>5,267 (13%)</b>  |
| #MLS Nets              | 0           | 2,425                | 0            | 2,536               |
| Run-Time               | -           | 20                   | -            | 15                  |
| Pwr                    | 1,539       | <b>1,523 (1.0%)</b>  | 1,157        | <b>1,152 (0.4%)</b> |
| Eff. Freq.             | 2,062       | <b>2,375 (15.2%)</b> | 1,517        | <b>1,582 (4.3%)</b> |

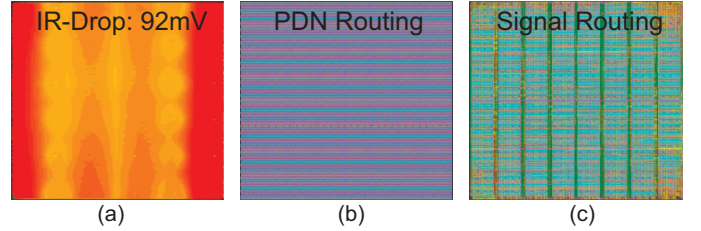


Fig. 9: (a)Hetero-MAERI 128PE IR-drop of 92mV. (b)PDN routing Layout. (c)Signal routing Layout.

tain IR-drop within 10% of  $V_{DD}$ . Specifically, MAERI 128PE reaches a 10% IR-drop due to its higher power consumption, whereas the A7 Dual-Core achieves a lower IR-drop of 2%, as shown in Figure 9(a). Figures 9(b)-(c) illustrate the sharing of top metal layer resources between PDN and Metal Layer Sharing (MLS) nets, demonstrating effective balancing of power delivery and signal routing. Level shifter (LS) power overhead remains comparable across designs, slightly increasing in SOTA and GNN-MLS due to additional MLS nets, resulting in a total power overhead of about 2–3%.

## V. CONCLUSION

This work presents GNN-MLS, a data-driven framework that enables efficient and effective cross-tier co-optimization through Metal Layer Sharing (MLS), significantly improving timing performance in 3D IC designs. Complementing this, our tailored Design-for-Test (DFT) solution ensures the testability of MLS connections, addressing a critical challenge in hybrid-bonded 3D ICs. Additionally, our power design supports heterogeneous integration with mixed-node configurations, facilitating robust experimentation in diverse 3D IC setups. Together, these contributions advance pseudo-3D flows toward true 3D IC performance by bridging the gap between 2D-based EDA tools and the demands of native 3D designs. This framework lays the groundwork for manufacturable 3D methodologies, marking a significant step toward fully optimized, high-density 3D ICs for commercial adoption.

## VI. ACKNOWLEDGEMENT

This work was supported by the Semiconductor Research Corporation under the JUMP 2.0 Center Program (CHIMES), Samsung Advanced Institute of Technology (SAIT) under the AI for Semiconductors Program, and the Ministry of Trade, Industry & Energy of South Korea (1415187652, RS-2023-00234159).

## REFERENCES

- [1] Peiyu Liao, Yuxuan Zhao, Dawei Guo, Yibo Lin, and Bei Yu. Analytical die-to-die 3-d placement with bistratal wirelength model and gpu acceleration. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(6):1624–1637, 2024.
- [2] Xueyan Zhao, Shijian Chen, Yihang Qiu, Jiangkai Li, Zhipeng Huang, Biwei Xie, Xingquan Li, and Yungang Bao. ipl-3d: A novel bilevel programming model for die-to-die placement. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023.
- [3] Jai-Ming Lin, Yu-Chien Lin, Hsuan Kung, and Wei-Yuan Lin. Hyplace-3d: A hybrid placement approach for 3d ics using space transformation technique. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8, 2023.
- [4] Nesara Eranna Bethur, Anthony Agnesina, Moritz Brunion, Alberto Garcia-Ortiz, Francky Catthoor, Dragomir Milojevic, Manu Komalan, Matheus Cavalcante, Samuel Riedel, Luca Benini, and Sung Kyu Lim. Hier-3d: A methodology for physical hierarchy exploration of 3-d ics. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 43(7):1957–1970, 2024.
- [5] Anthony Agnesina, Moritz Brunion, Jinwoo Kim, Alberto Garcia-Ortiz, Dragomir Milojevic, Francky Catthoor, Manu Perumkunnil, and Sung Kyu Lim. Power, performance, area and cost analysis of memory-on-logic face-to-face bonded 3d processor designs. In *2021 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*, pages 1–6, 2021.
- [6] Sai Surya Kiran Pentapati, Kyungwook Chang, Vassilios Gerousis, Rwik Sengupta, and Sung Kyu Lim. Pin-3d: A physical synthesis and post-layout optimization flow for heterogeneous monolithic 3d ics. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2020.
- [7] T. Kim et al. Multi-Stack Wafer Bonding Demonstration utilizing Cu to Cu Hybrid Bonding and TSV enabling Diverse 3D Integration. In *2021 IEEE 71st Electronic Components and Technology Conference (ECTC)*, pages 415–419, 2021.
- [8] Christopher Netzbund, Kevin Ryan, Yuji Mimura, Son Ilseok, Hirokazu Aizawa, Nathan Ip, Xuemei Chen, Hideyuki Fukushima, and Shinichi Tan. 0.5 m pitch next generation hybrid bonding with high alignment accuracy for 3d integration. In *2023 IEEE 73rd Electronic Components and Technology Conference (ECTC)*, pages 1100–1104, 2023.
- [9] Sai Pentapati and Sung Kyu Lim. Metal layer sharing: A routing optimization technique for monolithic 3d ics. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 30(9):1355–1367, 2022.
- [10] H. Park et al. Pseudo-3D Physical Design Flow for Monolithic 3D ICs: Comparisons and Enhancements. *ACM Trans. Des. Autom. Electron. Syst.*, 26(5), jun 2021.
- [11] Hyoukjun Kwon, Ananda Samajdar, and Tushar Krishna. MAERI: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. *ACM SIGPLAN Notices*, 53(2):461–475, 2018.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc.
- [13] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep graph infomax. *arXiv preprint arXiv:1809.10341*, 2018.

# Reinforcement Learning-Driven Window Selection for Enhanced Window-Based Rip-up and Reroute in Chip Detailed Routing

Yu-Chan Keng<sup>1</sup>, Yu-Chun Pai<sup>1</sup>, Wen-Hao Liu<sup>2</sup>, Haoxing Ren<sup>2</sup>, Danny Liu<sup>2</sup>, Rongjian Liang<sup>2</sup>, Mark Ho<sup>2</sup>, Anthony Agnesina<sup>2</sup>, and Yih-Lang Li<sup>1</sup>

<sup>1</sup>Department of Computer Science, National Yang Ming Chiao Tung University, Hsinchu, Taiwan; <sup>2</sup>NVIDIA Corporation, USA  
{kch772018, a0935796620}@gmail.com, {wenhliu, haoxingr, dannliu, rliang, chiatungh, aagnesina}@nvidia.com, ylli@cs.nctu.edu.tw

## ABSTRACT

With increasingly complex design rules and pin density in advanced technology nodes, achieving a violation-free layout has become more challenging, also making rip-up and reroute (RUR) the most runtime-intensive component of detailed routing. We propose a novel reinforcement learning (RL)-based approach to enhance the window-based RUR process. Our method features a dynamic window generation strategy that adjusts window size and position based on the distribution of design rule violations (DRV), enabling efficient targeting of congested areas. By leveraging the predictive capabilities of RL, our approach aims to minimize DRVs and achieve high-quality routing results. Experimental results demonstrate that our method outperforms the state-of-the-art detailed routers, TritonRoute, achieving a DRV-free solution, averagely improving wirelength by 0.07%, via count by 2.42%, and consuming almost the same average runtime.

## 1. INTRODUCTION

In the physical design flow, the detailed routing process is divided into two main stages based on different objectives: initial routing and RUR. In the first stage, connectivity for each net is established by following the routing guides provided by global routing. This stage typically requires less time, and is less likely to achieve a DRV-free result. To address remaining DRVs, the RUR scheme becomes crucial as detailed routing requires considering increasingly complex design rules and pin density.

Previous routing research employs a straightforward yet effective rip-up scheme that rips up the complete or partial routing of nets in the subsequent rerouting operation. This approach is known as the path-based RUR scheme [2, 3]. While the path-based scheme can improve outcomes through the negotiation-based RUR strategy [4], it can incur huge runtime costs, especially when the routing space is congested.

Another approach is to select a specific region or window containing DRVs and reroute partial or all of the wire segments within this window. This method is known as the window-based RUR scheme. A. B. Kahng et al. [1] introduced TritonRoute, an end-to-end detailed router that leverages the window-based approach to define routing regions. Fig. 1 presents a representative instance of DRV and window-based rerouting. The red rectangular region delineates the rerouting window, and the red pins denote the intersections between routed paths and the window's boundary. These intersection points and signal pins within the window serve as the connection targets for the rerouting operation. This method reduces routing problem size and can significantly reduce runtime of RUR process by rerouting non-overlapping windows in parallel using multithreading.

TritonRoute's window generation uses identical patterns, and

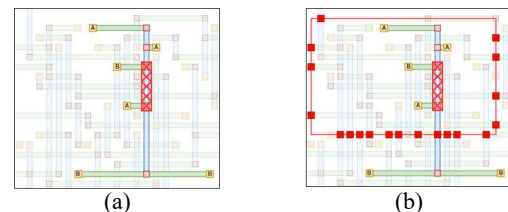


Fig. 1 (a) Example of DRV; (b) Example of window generation.

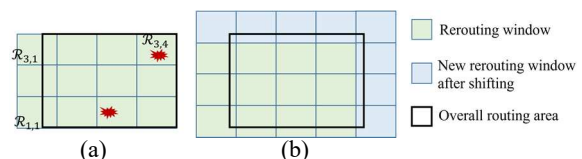


Fig. 2 (a) Partition an area into 3x4 rerouting windows; (b) Partition an area into 4x5 rerouting windows.

creates diverse rerouting regions by monotonic window shifting. Fig. 2(a) depicts a routing area partitioned into a 3x4 grid of rerouting windows, each encompassing 7x7 global cells (GCells). The rerouting procedure entails a systematic diagonal shift of each window towards the bottom-left corner until it exits the routing area. Upon the window shifting, a new window is formed to cover the newly exposed region, as illustrated in Fig. 2(b).

The monotonic window generation and shifting scheme depicts efficacy in the removal of DRVs. However, Fig. 2(a) shows the potential insufficiency in resolving DRVs. The DRVs situated within regions  $\mathcal{R}_{3,4}$  and  $\mathcal{R}_{1,3}$  cannot be rectified by the current partitioning scheme if they demand rerouting towards regions with locally lowest metal density, such as  $\mathcal{R}_{2,4}$  and  $\mathcal{R}_{1,2}$ . If the lower metal-density areas within  $\mathcal{R}_{3,4}$  and  $\mathcal{R}_{1,3}$  are located at the top and right areas, respectively, rerouting paths will be forced towards these areas, making DRV removal more challenging.

Although AI techniques have been successfully applied to various physical design problems—such as placement optimization, routing guide selection for detailed analog design routing, DRV removal for cell layouts, and obstacle-avoided Steiner tree generation [5-9]—there has yet to be a solution capable of addressing the general detailed routing problem due to its high complexity and difficulty. Our objective is to design an RL agent to optimize the selection of rerouting windows, with the aim of enhancing the quality of the detailed routing process.

The main contribution of this paper is the proposed RL-based dynamic window generation strategy, allowing flexible window sizing and positioning based on the distribution patterns of DRVs. We leverage RL's extensive learning experiences and predictive capabilities to evolving layout conditions, thereby expediting DRV resolution and minimizing the iterative process required to achieve a DRV-free layout. Experimental results show that,

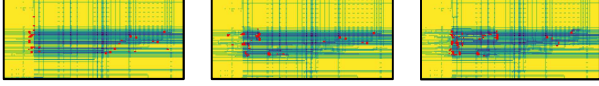


Fig. 3 The phenomenon of violation propagation (from left to right). The red markers represent the violations.

compared to TritonRoute, our approach resolves all DRCs, and improves wire length (WL) by 0.07% and via count by 2.42% with almost the same runtime.

The remainder of this paper is organized as follows: Section 2 introduces problem formulation and important previous works. Section 3 describes the design concepts. Section 4 outlines the RL framework design. Section 5 presents the experimental results, and Section 6 concludes the paper.

## 2. PRELIMINARY

### 2.1 Problem Formulation

The input to the problem includes a placed netlist, routing guides generated from global routing, and a set of design rule constraints. The primary goal is to successfully route all connections, while optimizing several key metrics such as DRV's, total wirelength, via count, and runtime.

### 2.2 Window-Based RUR Scheme

A. B. Kahng et al. [1] proposed TritonRoute, an open-source end-to-end detailed router that employs a window-based RUR scheme. In each iteration, the router segments the layout into discrete, non-intersecting windows, each spanning  $7 \times 7$  GCells. These window routings are subsequently conducted individually.

For each rerouting window, TritonRoute constructs a local grid graph. Maze routing is then applied to reroute selected subnets based on the window's Rip-up Mode (RM). In RM 0, only subnets with DRV's are ripped up and rerouted to reduce runtime and focus on DRV elimination; in RM 1, all subnets are ripped up to rebuild their topology, aiming for the opportunity to achieve a lower DRV count. RM 1 is utilized in preceding iterations of the TritonRoute to reduce wirelength, via count, and mitigate DRV's.

### 2.3 Reinforcement Learning

RL [13] is a machine learning paradigm where an agent learns to make decisions through interactions with an environment. The agent receives state observations from the environment, takes actions based on its policy, and receives rewards that indicate the quality of these actions.

#### 2.3.1 MaskablePPO

We employ Proximal Policy Optimization (PPO) [10], a SOTA RL algorithm known for its stability and reliability in continuous control tasks. PPO uses a clipped objective function to prevent excessive policy updates, making it particularly suitable for our window generation problem requiring consistent and stable learning. The algorithm updates its policy by optimizing:

$$L(\theta) = \mathbb{E}[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)], \quad (1)$$

where  $r_t(\theta)$  is the probability ratio between the new and old policies,  $A_t$  is the gain estimate, and  $\epsilon$  is the clipping parameter.

To handle invalid window configurations in our specific problem domain, we adopt MaskablePPO [11], an extension of PPO that incorporates action masking. This modification allows us to explicitly exclude invalid actions, such as windows that would exceed layout boundaries or violate minimum/maximum

size constraints during the policy sampling process. The action masking mechanism ensures that our agent only considers feasible window configurations.

#### 2.3.2 RL Network Architecture

The network architecture used in our work is a general RL framework provided by Stable Baselines3 [12]. The framework includes three essential components: the feature extractor, the policy network, and the value network.

#### 2.3.3 SE-ResNet

ResNet [14] addresses the gradient vanishing problem through shortcut connections, allowing for deeper neural network architectures. SE-ResNet enhances ResNet by integrating Squeeze-and-Excitation Networks (SE-Net) [15] into the original residual blocks. The "Squeeze" operation applies a global average pooling layer to condense spatial information into the channel dimension, capturing global insights across the feature map for each channel. Following this, the "Excitation" phase uses a fully connected network and a sigmoid function to assign importance to each channel, thus weighing them based on relevance.

A key advantage of SE-ResNet over standard ResNet is its adaptive channel weighting, which focuses on enhancing essential features while suppressing irrelevant ones. This selective emphasis on channel information, rather than purely spatial information, results in higher accuracy across a range of tasks, with only a modest increase in computation and parameters.

## 3. DESIGN CONCEPTS

### 3.1 DRV Propagation

The ideal RUR operation seeks to entirely eliminate all DRV's within a specified rerouting window. Nevertheless, in regions characterized by high congestion, the removal of existing DRV's can inadvertently result in the emergence of new DRV's within other areas that become congested after rerouting, termed *DRV propagation*. DRV propagation is a necessary process before all DRV's are eliminated. Fig. 3 shows the DRV propagation phenomenon. The figures, arranged from left to right, depict the routing iterations progressing from the early to the late stages.

Consider three regions:  $\mathcal{R}_1$ ,  $\mathcal{R}_2$ , and  $\mathcal{R}_3$ , arranged sequentially from left to right.  $\mathcal{R}_1$  is characterized by the presence of multiple DRV's, while  $\mathcal{R}_2$  and  $\mathcal{R}_3$  are characterized by high and low congestion, respectively. Rerouting paths through  $\mathcal{R}_2$  is likely to induce the emergence of new DRV's. If  $\mathcal{R}_1$ ,  $\mathcal{R}_2$ , and  $\mathcal{R}_3$  are encompassed within a single rerouting window, the complete elimination of all DRV's is feasible. However, if only  $\mathcal{R}_1$  and  $\mathcal{R}_2$  are included within the same rerouting window, and  $\mathcal{R}_3$  is excluded, rerouted paths are more likely towards low-congested regions, rather than  $\mathcal{R}_2$ , rendering DRV removal more intricate.

The dimension of the rerouting window constitutes another critical factor. A larger window possesses the potential to mitigate the aforementioned DRV propagation issue and eliminate all DRV's. Nevertheless, unnecessarily large windows can substantially increase computational runtime.

### 3.2 Layout-to-Tile Feature Transformation

#### 3.2.1 Tile-Based Rerouting Window Generation

We apply a tile-based data structure, wherein each tile depicts the smallest unit within the layout and stores local information,

including the number of DRV's and metal density. Rerouting windows are constructed by merging adjacent tiles, leveraging the tile information to guide the window generation process.

### 3.2.1.1 Tiling the layout

Initially, the layout is partitioned into tiles, with each tile possessing a width equivalent to half the width of a GCell. This finer-grained partitioning, in contrast to TritonRoute's window shifting distance of one GCell, affords a greater number of potential positions for window generation, thereby facilitating the creation of more diverse routing regions.

Each tile encompasses multiple layers, including metal and cut layers. For each layer within a tile, the number of DRV's and metal density are calculated. This results in a list of metrics that collectively represent the layout information for each layer within the tile. A tile is classified as violated if it contains at least one DRV on any of its constituent layers.

### 3.2.1.2 Building the rerouting window

Window generation starts by selecting a violated tile with the regions around it as the kernel window, subsequently extending the boundary of the current window in four directions: left, bottom, right, and top. Each extension expands the entire boundary of the current window, preserving its rectangular shape. The dimension of the rerouting window is determined by the agent based on a multitude of factors, including the number and severity of DRV's within the window, the metal density of the region, the current iteration number, the overall quality of the routing solution, and the computation time.

### 3.2.1.3 Window Objectives

The generated windows can be broadly classified into two objectives: propagation and elimination.

**Propagation:** In high-density regions, the removal of DRV's within a single window may prove challenging. A more effective approach involves enlarging the window to encompass regions of lower density or employing RM 1 to reconstruct the routing topology within the region. The reconstruction of the routing topology can also enhance the efficiency of DRV propagation, as the newly constructed topology will explore a more optimal solution to address the current DRV's.

**Elimination:** For windows possessing ample routing resources, the utilization of RM 0 to directly rip up and reroute violated nets is feasible, leading to the convergence of DRV's and a reduction in computational runtime.

A strong correlation exists between early and later iterations within the rerouting process, especially when addressing high-density regions. In early iterations, rerouting is applied to divide and disperse DRV's towards low-density regions, thereby creating more opportunities for their elimination in subsequent iterations.

Theoretically, adherence to the aforementioned policies can result in enhanced rerouting outcomes. However, the RL agent must acquire the ability to discern when to adhere to these policies and when to deviate from them, as there may exist exceptions wherein alternative strategies yield superior outcomes.

## 3.2.2 Calculating Tile Density

Two key pieces of information are considered: DRV density and metal density.

### 3.2.2.1 DRV density

Rather than employing a simple DRV count, we calculate the DRV density within each tile. This normalization is crucial for providing meaningful input to the agent, as raw DRV counts can exhibit significant variation across different datasets. Besides, a DRV characterized by a larger DRV area typically implies a more challenging eliminate process. Furthermore, by eschewing the differentiation between various types of DRV's, we simplify the problem formulation. Thus, the DRV area and density constitute more suitable metrics in comparison to raw DRV counts. The DRV density can be formulated as:

$$den_{vio} = \sum_{i \in S_{DRV}} area_{vio}(i) / \text{Tile Area}, \quad (2)$$

where  $S_{DRV}$  is the set of DRV's and  $area_{vio}(i)$  is the DRV area of the  $i^{th}$  DRV in  $S_{DRV}$ . This calculation is suitable for both metal layers and cut layers.

### 3.2.2.2 Metal density

Metal density encompasses net wire segments, pins, and obstacles within the metal layers, as well as vias within the cut layers. TritonRoute adopted a grid-based routing model. To calculate the metal density of a specific layer, the metal ratio of each track on that layer is computed, and these metal ratios are subsequently summed. The metal ratio of a track represents the proportion of the track occupied by metal and can be calculated as follows:

$$rat_M(t) = \sum_{mo \in \mathcal{MO}(t)} ml(mo) / tlen, \quad (3)$$

where  $rat_M(t)$  is the metal ratio of track  $t$ ,  $\mathcal{MO}(t)$  is the set of metal wire segments at track  $t$ ,  $ml$  is the metal length of a metal wire segment, and  $tlen$  is the track length. The metal density of a tile can then be formulated as:

$$den_M = \sum_{t \in \mathcal{T}} rat_M(t), \quad (4)$$

where  $\mathcal{T}$  is the track set in a tile.

To further refine our analysis, we subdivide metal wire segments into two categories: *fixed* and *non-fixed*. Fixed segments represent obstacle density and encompass objects such as pins and obstacles that remain unchanged during the RUR process. Non-fixed segments represent wire density and consist of routed wire segments that are subject to modification subsequent to the rerouting process.

In the cut layer, the available routing space can be determined by the total number of valid via candidates. A via candidate is a track intersection point between adjacent metal layers. The via density can be quantified as follows:

$$den_{via} = \#Vias / \#VVC, \quad (5)$$

where  $\#VVC$  is the total number of valid via candidates.

## 3.3 Overview of the Detailed Routing Flow

In the preprocessing stage, we perform TritonRoute's initial procedures, including the parsing of input data, the preparation of routing data, the generation of pin access points, and the track assignment. After that, we construct the tile data structure and initialize the tile information. We then apply TritonRoute to obtain the first-iteration routing result that guarantees net connectivity (i.e., no open nets). Subsequent to the initial routing, we identify violated tiles and sort them in ascending order of density, from low-density regions to high-density regions. This density-based sorting strategy is designed to strategically release routing space around congested areas during the early iterations.

In each routing iteration, violated tiles are sequentially selected and utilized as kernel windows. For each kernel window, our agent generates a list of non-overlapping windows and selects a rip-up mode for each window. TritonRoute then completes the rerouting of each window. Any rerouting attempts that fail to meet TritonRoute's constraints are discarded rather than committed to the layout.

Subsequent to the completion of window processing in each iteration, the tile information is updated, which entails recalculating DRV densities and non-fixed metal densities. The termination conditions are then evaluated. The detailed routing stage concludes when either all DRVs are resolved or the total number of iterations attains the predetermined upper limit.

## 4. RL-BASED METHODOLOGY

### 4.1 RL Methodology Setting

#### 4.1.1 Environmental State

We employ a CNN-based network architecture to process the layout information derived from our tile-based representation. Rather than processing the entire layout, we focus on a 64×64 tile region centered around the target violated tile as our environmental state. This localized approach is justified by two key factors: Firstly, DRV propagation typically affects only nearby regions, rendering local information sufficient for effective window generation decisions. Secondly, this fixed-size state representation enables our method to handle layouts of varying dimensions uniformly.

To provide a comprehensive representation of the layout, we adopt a 3D environmental state. This 3D state is constructed by considering the layout in three dimensions: width, height, and layer. Each layer, such as metal or cut, has its own density map.

Key components of the environmental state contain metal layer density map (3D), cut layer density map (3D), and auxiliary information map (2D). In metal layer density map, DRV density indicates the concentration of DRVs. Obstacle density represents the density of fixed objects like pins and obstacles. Wire density reflects the density of routed wires. In cut layer density map, DRV density indicates the concentration of DRVs. Obstacle density represents the density of fixed objects like vias in the power network. Via density reflects the density of routed vias. In auxiliary information map, rerouted count map tracks the number of rerouting attempts in a region. RM 0 failure count map records the number of failed RM 0 attempts in a region. RM 1 failure count map records the number of failed RM 1 attempts in a region. Window position map indicates the current position of the rerouting window on the layout.

By incorporating these components, the RL agent can make informed decisions about window generation and RUR strategies, leading to improved routing quality.

#### 4.1.2 Action

To generate a rerouting window, the agent must sequentially decide on window boundary extensions (up, down, left, right) and select a rip-up mode (RM 0, RM 1). This action space enables the agent to configure the window and its associated routing strategy.

#### 4.1.3 Reward

To optimize these actions effectively, we have designed separate reward functions for each of the two types of actions. For

the window generation step, the reward function encourages optimal window configuration. Once the agent selects a rip-up mode, the window immediately proceeds with detailed routing, linking the detailed routing step directly to the rip-up mode selection. This structure allows each action type to be guided by its respective reward, promoting effective decision-making in both window sizing and routing strategy.

#### 4.1.3.1 Window generation reward

A DRV is more likely to be eliminated if it is not located near the rerouting window's boundary. This is because in such cases, the DRV has four potential directions to allocate rerouting paths, increasing the chances of successful eliminate. To address this issue, we impose a constraint on the window's aspect ratio and designed a window size reward  $rw_{ws}$  for extension actions. We define window size reward as:

$$rw_{ws} = \begin{cases} 0.05, & \text{if } 0.6 \leq ar \leq 1.7 \text{ and } (wid_w \leq 15 \text{ or } hgt_w \leq 15) \\ 0.02, & \text{if } 0.6 \leq ar \leq 1.7 \text{ and } (wid_w > 15 \text{ or } hgt_w > 15) \\ 0, & \text{otherwise} \end{cases} \quad (6)$$

where  $ar$  is the aspect ratio of the window,  $wid_w$  is the width of the window, and  $hgt_w$  is the height of the window. This reward encourages the agent to extend the window toward a nearly square shape, aiming to position DRVs closer to the center of the window. To prevent the agent from excessively favoring maximum window extensions, the reward diminishes when the window size exceeds 15. However, the design does not strictly enforce a square window to allow some flexibility in handling specific scenarios that may require different patterns.

#### 4.1.3.2 Detailed routing reward

In this work, the main objective is to eliminate DRVs as much as possible. Therefore, the reward design considers the remaining DRVs after a window rerouting has been completed. We define the DRV reward as:

$$rw_{vio} = (vio_{ini} - vio_{rem}) / vio_{rem}, \quad (7)$$

where  $vio_{ini}$  is the initial number of DRVs in the window before rerouting and  $vio_{rem}$  is the remaining number of DRVs in the window after rerouting.

Considering the two different rip-up modes, RM 0 takes less runtime than RM 1 due to the number of rerouted nets. To suppress the agent's preference for selecting RM 1, we first define the rerouted nets proportion ( $rnp$ ):

$$rnp = \#Net_{reroute} / \#Net_{total}, \quad (8)$$

where  $\#Net_{reroute}$  is the number of reroute nets in the window, and  $\#Net_{total}$  is the total number of nets in the window. While RM 1 always reroutes all nets ( $rnp = 1$ ), RM 0 reroutes a subset of nets. To account for potential runtime differences, especially in windows with fewer nets, we define the penalty for the rip-up mode as follows:

$$penalty_{RM} = rnp * (wd_{window} / \max_{i \in \text{tile}}(wd_{tile}(i))), \quad (9)$$

where  $wd_{tile}(i)$  is the wire density value of tile  $i$ . The ratio of the window's density to the maximum tile density serves as an indicator of the window's congestion level. In regions characterized by high congestion, RM 1, which involves the sequential rerouting of all nets, incurs a higher penalty due to increased computational complexity. Finally, the total reward of detailed routing is defined as:

$$rw_{dr} = \begin{cases} \alpha \cdot rw_{vio} + \beta \cdot penalty_{RM}, & \text{if } dr \text{ success} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

## 4.2 Flow Optimization

The SE-ResNet architecture integrates Squeeze-and-Excitation (SE) blocks, enabling the agent to assign adaptive weights across feature channels and make stage-aware decisions. In our method, the 3D state maps capture critical layout information, including DRV and metal density, which are intrinsically interrelated, while the 2D auxiliary state map provides essential routing iteration information. The SE blocks facilitate the integration of these diverse channels, allowing the agent to better capture cross-channel dependencies. This capability enhances the agent’s ability to derive globally optimized solutions, effectively balancing DRV resolution and routing quality.

In early stages, the SE-ResNet agent tends to select larger windows and RM 1. This strategy efficiently resolves dense DRVs and mitigates congestion in critical area. As the routing process progresses into the mid-to-late stages, the SE-ResNet agent shifts its focus to smaller window sizes and RM 0, thus optimizing runtime and enhancing routing efficiency.

This dynamic adaptation demonstrates SE-ResNet’s ability to balance short-term performance with long-term routing quality, providing a holistic approach to resolving DRVs while improving overall routing outcomes.

### 4.3 Training Workflow

Each layout is transformed into a training data instance by:

- a. *Initial Routing*: Perform the preprocessing stage and the initial iteration of TritonRoute to find an initial routing solution.
- b. *DRV Prioritization*: Identify and prioritize violated tiles based on the density-based sorting strategy outlined in Section 3.3.

Each training episode includes:

- a. *Target Tile Selection*: Select a violated tile from the prioritized list as the target tile for the current episode.
- b. *Environmental State Generation*: Provide the RL agent with an environmental state encompassing the layout information centered around the target tile.
- c. *Window Generation*: Guide the agent to generate a window around the target tile through multiple decision-making steps.
- d. *Detailed Routing*: Execute the detailed routing process within the generated window.
- e. *Reward Calculation*: Compute a reward based on the quality of the routing solution within the window.
- f. *Episode Termination*: Conclude the episode and transition to the next target tile or a new layout.

A new layout is introduced as training data once all DRVs in the current layout have been resolved or the total number of iterations attains the predetermined upper limit. By iteratively training the agent on a sequence of layouts, the agent learns to effectively generate windows and select appropriate RUR strategies to improve routing quality.

## 4.4 Network Architecture

Fig. 4 depicts the network architecture used in this work. The feature extractor uses a CNN-based network, SE-ResNet34, to process three individual environmental states. Due to the nature differences between metal layers and cut layers, we do not combine metal layer density and cut layer density as a single environmental state. Instead, we use different networks to process them. The 3D metal layer state (wire features) and 3D cut layer state (via features) are processed through two SE-ResNet networks with 3D convolutional layers. A mean function is then

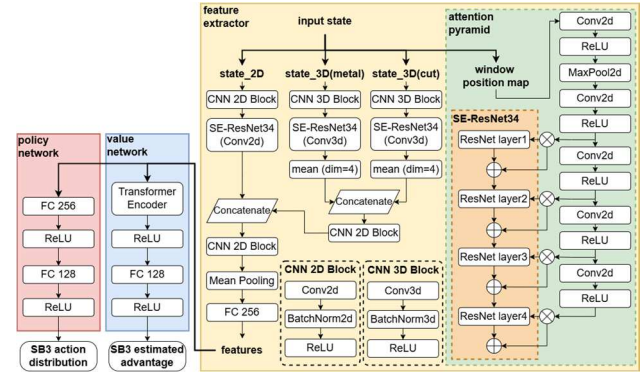


Fig. 4. The network architecture of the RL agent.

used to transform these two features into 2D representations, which are subsequently fused and passed through a convolutional layer to obtain integrated 3D features with 2D dimensions. On the other hand, the 2D states are processed using a SE-ResNet with 2D convolutional layers. The 2D features are concatenated with the integrated 3D features and passed through a final convolutional layer, followed by global pooling and a fully connected layer, to obtain the final feature representation.

A unique aspect of the feature extractor is the window position network. The window position map (one channel of the 2D state) is passed into an independent network consisting of four convolutional layers. The output of each layer becomes an attention feature with window position information. This attention feature is integrated into each SE-ResNet layer through element-wise multiplication and addition with the corresponding SE-ResNet output feature. The reason for adding window position information is that while the agent needs information about the surrounding region during window boundary extension, it only needs to consider regions inside the generated window when determining the rip-up mode.

In the policy and value network, we employ fully connected layers to produce the action distribution used to select an action. In the value network, a fully connected layer is alternated with a transformer encoder, enhancing the network's capacity to model complex value representations and strengthen the advantage estimation. After the outputs of the policy and value networks are generated, the Stable Baselines3 internal functions use the action distribution to select an action and use a final fully connected layer in the value network to estimate the advantages.

## 4.5 Detailed Routing Parallelization

In the training stage, we have to process the detailed routing of each window sequentially because of the reward calculation and returning in one episode. To achieve parallelization, we can sequentially generate multiple windows and reroute them concurrently. This approach works because we need to provide the next environmental state at each step during training, which is not necessary in the testing stage.

Another bottleneck for runtime is the agent inference time. For each step, we must pass through the entire network once, and the accumulated inference time can become considerable as the layout size increases. To address this, we utilize a batch prediction method. First, we generate multiple kernel windows on the layout. We then place each window state into a batch and pass them through the model simultaneously, which leverages GPU parallelization and generates all windows concurrently.

TABLE 1. COMPARISON BETWEEN OURS AND TRITONROUTE

|       | DRV  |    | Runtime      | Wirelength   | Via          |
|-------|------|----|--------------|--------------|--------------|
|       | Ours | TR | (Ours-TR)/TR | (Ours-TR)/TR | (Ours-TR)/TR |
| 18_1  | 0    | 0  | -11.63%      | -0.09%       | -0.23%       |
| 18_2  | 0    | 0  | -27.51%      | -0.06%       | -0.89%       |
| 18_3  | 0    | 0  | -2.90%       | -0.03%       | -0.69%       |
| 18_4  | 0    | 9  | -36.16%      | -0.09%       | -0.49%       |
| 18_5  | 0    | 0  | 7.05%        | -0.13%       | -3.68%       |
| 18_6  | 0    | 0  | 3.26%        | -0.07%       | -3.82%       |
| 18_7  | 0    | 0  | -3.31%       | -0.09%       | -3.41%       |
| 18_8  | 0    | 0  | 4.14%        | -0.03%       | -3.42%       |
| 18_9  | 0    | 0  | 9.38%        | -0.04%       | -3.62%       |
| 18_10 | 0    | 8  | 63.80%       | -0.08%       | -3.90%       |
| Avg.  |      |    | 0.61%        | -0.07%       | -2.42%       |

## 5. EXPERIMENTAL RESULTS

The proposed window selection framework was implemented by modifying TritonRoute's open-source code [1] in C++, while the RL agent was developed using the Stable Baselines3 [12] RL framework in Python. All experiments were conducted on a Linux platform with a 2.1GHz Intel Xeon Gold 5218R CPU, 256 GB RAM, and an NVIDIA GeForce RTX 3090 GPU.

### 5.1 Training and Testing Datasets

Our experiments used the ISPD-2018 benchmark suite [16], which includes 10 test cases. To adapt the ISPD benchmarks for our testing dataset, we employed AutoDMP [17] to perform cell replacement and CUGR [18] to perform global routing on the benchmarks. The training dataset consists of 15 modified layouts derived from four ISPD benchmarks (ispd18\_test2, test3, test4, and test5). Each replaced layout retains the same standard cell count and fixed macro positions, creating diverse placements that serve as identical layouts for training purposes. We preprocessed all training data using TritonRoute. The results show that 7 training data points achieved DRV-free routing with TritonRoute, while 8 data points exhibited DRVs. The testing dataset contains all 10 ISPD-2018 benchmark test cases. These benchmarks were processed similarly to the training dataset but were reserved exclusively for evaluating the framework's performance and generalizability on unseen layouts.

### 5.2 Detailed Routing Results

Table 1 presents a comparison between our method and TritonRoute for detailed routing on the ISPD-2018 benchmarks. Metrics such as DRV, wirelength, and via count were calculated using TritonRoute. Runtime was measured starting from the end of TritonRoute's initial iteration (iteration 0).

In the ISPD-2018 benchmarks, certain test cases (notably 18\_3, 18\_4, and 18\_10) are considered challenging. TritonRoute is unable to achieve DRV-free results on 18\_4 and 18\_10. As we increase the rerouting iteration number of TritonRoute, the remaining DRV numbers of them remain unchanged. In contrast, our method successfully achieves DRV-free routing on all ISPD-2018 test cases, demonstrating that the proposed RL agent has more flexibility to handle complex DRVs in these benchmarks.

For runtime, our method outperformed TritonRoute in five test cases but was slower in others, with an average runtime increase of just 0.61%, effectively resulting in a near tie. Our runtime performance is measured as the total runtime of the routing flow, including the detailed routing time, the environment state update time, and the agent inference time required for generating actions.

TABLE 2. COMPARISON BETWEEN SE-RESNET AND RESNET

|       | Early Stage |      |        |      |       | Middle Stage |      |        |      |
|-------|-------------|------|--------|------|-------|--------------|------|--------|------|
|       | SE-ResNet   |      | ResNet |      |       | SE-ResNet    |      | ResNet |      |
| Iter. | RM_0        | RM_1 | RM_0   | RM_1 | Iter. | RM_0         | RM_1 | RM_0   | RM_1 |
| 3     | 8           | 389  | 378    | 20   | 12    | 157          | 188  | 341    | 37   |
| 4     | 34          | 362  | 387    | 19   | 13    | 175          | 161  | 344    | 33   |
| 5     | 28          | 367  | 361    | 40   | 14    | 163          | 156  | 343    | 29   |
| 6     | 64          | 315  | 366    | 34   | 15    | 199          | 122  | 346    | 36   |
| 7     | 67          | 307  | 335    | 31   | 16    | 208          | 100  | 336    | 40   |

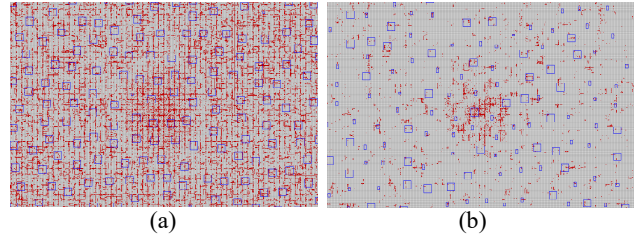


Fig. 5. Window selected by SE-ResNet agent and DRV tiles in the (a) early stage and (b) middle stage of ISPD 18\_10.

Regarding quality metrics, our approach bettered both wirelength and via count by an average improvement of 0.07% and 2.42%, respectively. These results show that our method can produce cleaner routing results with better wirelength and via count at a little cost of runtime.

### 5.3 A Comparative Analysis of SE-ResNet

Table 2 highlights the distinct rip-up mode selection patterns for the 18\_10 testcase across different iteration stages. ResNet-based agent shows a consistent preference, favoring a single rip-up mode throughout the iterations. In contrast, SE-ResNet-based agent shows a more dynamic approach, initially selecting RM 1 more frequently in the early stages, before gradually shifting to favor RM 0 as it progresses through the mid-to-late stages.

For the window size selection, ResNet-based agent tends to select smaller window sizes, while the SE-ResNet-based agent shows a more balanced distribution in window size choices. In Fig. 5, red and blue areas represent DRV tiles, and the generated windows, respectively. As iterations progress, the SE-ResNet-based agent increasingly selects smaller window sizes.

Based on the experimental results, it is evident that SE-ResNet leverages the SE-block to assign adaptive weights across channels, enabling the agent to learn that optimal window sizes and rip-up modes vary at different stages. This adaptation allows the agent to resolve DRVs while simultaneously improving routing quality. In the early stages, SE-ResNet's preference for larger windows and the selection of RM 1, which involves ripping up and rerouting all nets within the window, contributes to an increase in runtime. However, in the mid-to-late stages, the agent shifts to smaller windows and more frequent use of RM 0, effectively controlling runtime within manageable limits.

## 6. CONCLUSIONS

We proposed a novel framework for adaptive window generation and RM selection for detailed routing optimization. Our method, leveraging a SE-ResNet-based agent, dynamically generates and refines routing windows by merging tiles based on routing information. Experimental results on the ISPD-2018 benchmarks demonstrate that our method achieves superior routing quality and efficiency, and more effective capability of removing DRVs.

## 7. REFERENCES

- [1] A. B. Kahng, et al., "TritonRoute: The Open-Source Detailed Router," *IEEE TCAD*, vol. 40, no. 3, pp. 547-559, 2021.
- [2] G. Chen, et al., "Dr. cu: Detailed routing by sparse grid graph and minimum-area-captured path search," *IEEE TCAD*, vol. 39, no. 9, pp. 1902-1915, 2020.
- [3] H. Li, et al., "Dr. CU 2.0: A Scalable Detailed Routing Framework with Correct-by-Construction Design Rule Satisfaction," in *IEEE/ACM ICCAD*, pp. 1-7, 2019.
- [4] W.-H. Liu, et al., "NCTU-GR 2.0: multithreaded collision-aware global routing with bounded-length maze routing," *IEEE TCAD*, vol. 32, no. 5, pp. 709-722, 2013.
- [5] A. Mirhoseini, et al., "Chip placement with deep reinforcement learning," arXiv:2004.10746 [cs.LG], 2020.
- [6] H. Chen, et al., "Reinforcement learning guided detailed routing for custom circuits," in *ACM/IEEE ISPD*, pp. 26-34, 2023.
- [7] P.-Y. Chen, et al., "A reinforcement learning agent for obstacle-avoiding rectilinear steiner tree construction," in *ACM/IEEE ISPD*, pp. 107-115, 2022.
- [8] J. Liu, et al., "Rest: Constructing rectilinear steiner minimum tree via reinforcement learning," in *ACM/IEEE DAC*, pp. 1135-1140, 2021.
- [9] H. Ren, et al., "Invited- nvcell: Standard cell layout in advanced technology nodes with reinforcement learning," in *ACM/IEEE DAC*, pp. 1291-1294, 2021.
- [10] J. Schulman, et al., "Proximal policy optimization algorithms," arXiv:1707.06347 [cs.LG], 2017.
- [11] S. Huang, et al., "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," arXiv:2006.14171 [cs.LG], 2022.
- [12] A. Raffin, et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, 2021.
- [13] R. Sutton, et al., "Reinforcement Learning: An Introduction," MIT Press, 1998.
- [14] K. He, et al., "Deep Residual Learning for Image Recognition," arXiv:1512.03385 [cs.LG], 2015.
- [15] J. Hu, et al., "Squeeze-and-Excitation Networks," arXiv:1709.01507 [cs.LG], 2019.
- [16] S. Mantik, et al., "ISPD 2018 initial detailed routing contest and benchmarks," in *ISPD*, pp. 140-143, 2018.
- [17] A. Anthony, et al., "AutoDMP: Automated DREAMPlace-based Macro Placement," in *ISPD*, pp. 149-157, 2023.
- [18] J. Liu, et al., "CUGR: Detailed-Routability-Driven 3D Global Routing with Probabilistic Resource Model," in *ACM/IEEE DAC*, pp. 1-6, 2020.